

SAGE — *Smart Adaptive Gating Engine for Cost-Efficient Code-Model Selection*

Project Proposal for UCS503P
Submitted to: Ms. Paramveer Sidhu

Thapar Institute of Engineering and Technology

Krishna Gautam Roll No: 1024030733 kgautam_be24@thapar.edu	Prateek Yadav Roll No: 1024031149 pyadav_be24@thapar.edu	Harshvardhan Roll No: 1024030744 hharshvardhan_be24@thapar.edu
---	---	---

August 13, 2026

1 Higher-order goal

... secondary framing

This project aims to improve the efficiency of AI-assisted software development by automatically selecting an appropriate code-generation model for each coding request. While modern large language models can solve a wide range of programming tasks, using the strongest model for every request can result in unnecessary cost, latency, and computational usage. The immediate engineering objective is to translate *cost-efficient and quality-aware model selection* into a measurable and deployable software solution.

2 Time-to-value

... primary heuristic

- We will deliver meaningful increments quickly by first implementing a baseline router that classifies coding prompts into difficulty levels and maps them to suitable Claude models.
- We will prioritize measurable experimentation through short iterations, comparing the routing system against a baseline that always uses a stronger model.
- Success is defined by measurable improvement in cost and latency while maintaining acceptable coding quality and correctness.

3 Problem Statement

AI coding assistants increasingly rely on powerful large language models to generate, debug, explain, and refactor software. However, not every coding request requires the same level of reasoning capability. A simple syntax question and a complex multi-file debugging task may both be sent to the same expensive model even though their computational requirements are very different. Common pain points include:

- **Over-provisioning:** simple coding tasks may consume the capabilities and cost of a high-end reasoning model unnecessarily.

- **Cost inefficiency:** repeatedly using a stronger model increases token consumption and overall inference cost for workloads containing many easy or moderate tasks.
- **Latency:** stronger models may introduce additional response time when a faster model would have been sufficient.
- **Difficult model selection:** users and applications generally have to decide manually which model should handle a particular coding request.
- **Quality-risk tradeoff:** selecting a cheaper model can reduce cost, but selecting one that is too weak for a difficult task can reduce correctness and usefulness.

Why this matters now (contextual relevance): As AI-assisted programming becomes a regular part of software development, efficient model utilization becomes increasingly important. A routing layer that can automatically match task complexity with model capability can reduce unnecessary inference cost without simply sacrificing output quality.

4 Proposed Solution

... Software Proposition

4.1 Overview

Build a software-based “gating engine” that analyzes a coding request before sending it to a Claude model. The system will estimate the complexity of the task, select an appropriate model tier, forward the request, and record the result for evaluation.

- **Coding prompt intake:** accept a programming request together with optional context such as language, code snippets, files, and task type.
- **Prompt analysis:** extract signals related to coding difficulty, reasoning requirements, context size, debugging complexity, and expected output.
- **Gating engine:** assign the request to an appropriate difficulty/model tier.
- **Model execution:** send the request to the selected Claude model through the appropriate API.
- **Evaluation and logging:** record routing decisions, token usage, latency, and task outcomes for comparison against baseline strategies.

4.2 Core workflow

1. User submits a coding prompt to the SAGE gateway.
2. The system extracts relevant task characteristics, such as task type, code/context size, reasoning depth, and expected implementation complexity.
3. The gating layer assigns a difficulty score or difficulty class such as Easy, Moderate, Hard, or Very Hard.
4. The routing policy maps the difficulty class and confidence to a suitable Claude model.
5. The selected model generates the response.
6. The system records cost-related and quality-related metrics and can identify cases where a stronger model would have been more appropriate.

4.3 Operational constraints for an educational setting

- The initial implementation will focus on coding prompts rather than attempting to route every possible natural-language task.
- The system will use commercially available model APIs and will not attempt to train a large language model from scratch.
- Routing decisions should be lightweight so that the cost and latency introduced by the gating layer do not outweigh the savings achieved through model selection.
- The project will emphasize measurable evaluation, reproducibility, and system correctness rather than claiming a new state-of-the-art language model.

5 Solution Approach

... Engineering focus

This is an engineering-focused AI systems project. The main focus is reliable model routing, measurable cost-quality optimization, and evaluation rather than training a new foundation model.

5.1 Prompt difficulty classification

... routing signal

The gating engine will classify coding requests using a combination of observable task characteristics:

- **Task type:** explanation, code generation, debugging, optimization, refactoring, testing, or architecture.
- **Reasoning requirement:** whether the task requires straightforward generation or multi-step reasoning and problem solving.
- **Context complexity:** approximate amount and structure of code or repository context supplied to the model.
- **Implementation complexity:** number of components, dependencies, edge cases, or interacting requirements.
- **Routing confidence:** confidence in the selected difficulty tier, allowing uncertain requests to be routed more conservatively.

5.2 Gating and model-selection policy

The first version will use a transparent routing policy that maps difficulty tiers to model tiers. Easy requests will be directed toward a faster and lower-cost model, moderate requests toward a balanced model, and difficult requests toward a stronger reasoning-capable model.

The policy will be evaluated rather than assuming that the strongest model is always optimal. If the system detects low confidence or a task that exceeds the expected capability of the selected model, a fallback mechanism can escalate the request to a stronger model.

5.3 Evaluation architecture

... quality-aware routing

A separate evaluation layer will compare routing decisions against baseline strategies. The evaluation layer will consider:

- correctness of generated code,

- test-case pass rate where executable tests are available,
- token consumption and estimated inference cost,
- response latency,
- routing accuracy,
- and the percentage of tasks successfully handled without escalation.

This allows the project to measure whether lower-cost routing actually preserves useful coding performance.

5.4 System architecture

... gateway-based solution

A typical architecture will contain the following components:

- **Client/API layer:** accepts coding prompts and optional code or repository context.
- **Prompt analyzer:** extracts task and complexity signals.
- **Gating engine:** calculates the difficulty tier and routing confidence.
- **Model router:** maps the decision to the selected Claude model and forwards the request.
- **Evaluation and logging layer:** stores routing decisions, latency, usage, and evaluation results.
- **Dashboard or reporting layer:** presents model utilization, cost savings, quality, and routing performance.

5.5 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run unit and integration tests on every change.
- Validate routing rules and API integrations before deployment.
- Produce repeatable deployment artifacts for a staging environment.
- Enable safe and frequent improvements to the routing policy and evaluation pipeline.

6 Evaluation Criterion

... measurable and attributable

We define success using measurable metrics tied directly to model-selection efficiency and coding quality.

6.1 Primary evaluation metric

Cost-quality efficiency: the primary objective is to reduce average model inference cost while maintaining coding quality close to a strong-model baseline.

- **Target:** demonstrate a measurable reduction in average inference cost compared with always routing requests to the strongest available model, without a significant decrease in correctness.
- **Attribution:** compare identical benchmark prompts under the baseline and SAGE routing policies and record model usage, token consumption, latency, and evaluation scores.

6.2 Secondary evaluation metrics

- **Code correctness:** percentage of generated solutions that pass predefined tests where executable test cases are available.
- **Quality retention:** compare SAGE responses against responses generated by a strong-model baseline using task-specific evaluation criteria.
- **Cost reduction:** percentage reduction in estimated model inference cost relative to the always-strong-model baseline.
- **Latency reduction:** average and median response-time improvement resulting from routing simpler tasks to faster models.
- **Routing accuracy:** percentage of tasks for which the selected model tier is judged sufficient for the task.
- **Escalation rate:** percentage of requests initially routed to a lower tier that require escalation to a stronger model.

6.3 Benchmark validation plan

... high-level

- Build a representative benchmark of coding prompts covering explanation, generation, debugging, refactoring, algorithms, testing, and multi-step engineering tasks.
- Divide prompts into difficulty categories using clearly defined criteria and, where possible, human validation.
- Run the same benchmark using an always-strong-model baseline and the SAGE routing policy.
- Compare correctness, cost, latency, model utilization, and escalation behavior.
- Analyze failure cases where the router selected a model that was too weak or unnecessarily strong.

7 Scalability

... with foresight

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The routing architecture should support increasing numbers of coding requests by:
 - keeping the routing layer lightweight,
 - separating prompt analysis from model execution,
 - using asynchronous request handling where appropriate,
 - and storing evaluation data independently from the request path.
2. **Operationally deployable (practically):** The system should be deployable as a gateway service:
 - containerized backend,
 - managed database or logging store,
 - configurable model-provider credentials,
 - and CI/CD pipeline for staging and production.

8 Engine availability heuristic

A mature set of software components and AI services is available to implement the proposed system:

- Backend framework for exposing the routing API.
- Claude model APIs for model execution.
- Standard machine-learning or rule-based techniques for prompt classification and scoring.
- Database or structured storage for prompts, routing decisions, evaluation results, and usage data.
- Testing frameworks for unit, integration, and benchmark evaluation.
- Containerization and CI/CD tooling for repeatable deployment.

We will use these mature components to focus on routing strategy, evaluation, correctness, and measurable cost-quality optimization rather than building model infrastructure from scratch.

9 Project scope and deliverables

... iteration-friendly

9.1 Initial deliverable

... first iteration

- Coding prompt gateway/API.
- Initial difficulty classification scheme with transparent routing rules.
- Integration with multiple Claude model tiers.
- Basic logging of selected model, latency, and token usage.
- A small benchmark dataset covering multiple coding difficulty levels.
- Baseline comparison against always using a strong model.
- CI: build, unit tests, linting, and routing tests.

9.2 Subsequent deliverables

- Improved difficulty classifier based on benchmark results.
- Confidence-aware routing and automatic escalation for uncertain or failed tasks.
- Expanded evaluation suite for code correctness, cost, latency, and quality retention.
- Dashboard/report showing model distribution, estimated cost savings, routing accuracy, and failure cases.
- Support for richer repository and multi-file context analysis.
- Optimization of the routing policy using observed evaluation results.

10 Risks and mitigations

- **Incorrect difficulty classification:** define explicit classification criteria, maintain a benchmark set, and use confidence-aware fallback to a stronger model.
- **Quality degradation:** evaluate routed responses against a strong-model baseline and prioritize correctness over cost when the router is uncertain.
- **Routing overhead:** keep the gating layer lightweight so that analysis cost and latency do not eliminate the benefits of model selection.
- **Changing model capabilities or pricing:** keep model-selection policies configurable rather than hard-coding assumptions about a particular model.
- **Evaluation ambiguity:** define benchmark tasks and objective metrics such as test pass rate, token usage, latency, and estimated cost before comparing routing strategies.
- **API availability and limits:** use a staging environment, implement error handling, and maintain clear separation between routing logic and provider integration.

11 Summary

This project proposes SAGE, a deployable gating engine for cost-efficient selection of Claude models for coding tasks. It addresses the practical problem of using unnecessarily powerful models for simple requests while avoiding quality loss on difficult programming tasks. The project meets the course criteria by emphasizing rapid engineering iterations, CI/CD, measurable evaluation, and scalability readiness.

The key contribution is not a new language model, but a measurable model-routing system that attempts to achieve a better tradeoff between coding quality, inference cost, and response latency. By comparing SAGE against an always-strong-model baseline, the project will demonstrate whether intelligent model selection can provide meaningful efficiency gains while preserving useful coding performance.